

# Base R Regex Functions

| PATTERN / SYNTAX                                                             | DESCRIPTION & EXAMPLE                                                                                                                                                                                                      |
|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>grepl(pat, x, perl = FALSE, fixed = FALSE, ignore.case = FALSE)</code> | Returns logical vector (TRUE/FALSE) indicating if pattern was matched in each string<br><br>EXAMPLE: <code>grepl("^\\d+", c("123", "abc"))</code> → <code>c(TRUE, FALSE)</code>                                            |
| <code>grep(pat, x, value = FALSE, perl = FALSE)</code>                       | Returns integer indices of elements matching pattern, or matched string values if value = TRUE<br><br>EXAMPLE: <code>grep("^a", c("apple", "banana", "avocado"), value = TRUE)</code> → <code>c("apple", "avocado")</code> |
| <code>sub(pat, repl, x, perl = FALSE)</code>                                 | Replaces first occurrence of pattern in each string element with replacement string<br><br>EXAMPLE: <code>sub("\\d+", "X", "item 10 and 20")</code> → <code>"item X and 20"</code>                                         |
| <code>gsub(pat, repl, x, perl = FALSE)</code>                                | Replaces all occurrences of pattern in each string element with replacement string (global)<br><br>EXAMPLE: <code>gsub("\\s+", "-", "hello big world")</code> → <code>"hello-big-world"</code>                             |
| <code>regexpr(pat, x, perl = FALSE)</code>                                   | Finds first match per element; returns integer vector of start indices with 'match.length' attribute<br><br>EXAMPLE: <code>m &lt;- regexpr("\\d+", "id: 1042"); m</code> → <code>5</code> (match.length = 4)               |
| <code>gregexpr(pat, x, perl = FALSE)</code>                                  | Finds all matches in each element; returns list of integer vectors with match offsets and lengths<br><br>EXAMPLE: <code>gregexpr("\\b\\w+\\b", "foo bar baz")</code>                                                       |
| <code>regexec(pat, x, perl = FALSE)</code>                                   | Finds first match and capturing groups; returns list of indices and lengths for each group<br><br>EXAMPLE: <code>regexec("([a-z]+):([0-9]+)", "port:8080")</code>                                                          |
| <code>regmatches(x, m) / regmatches(x, m) &lt;- value</code>                 | Extracts or replaces substrings from x matching positions returned by regexpr, gregexpr, or regexec<br><br>EXAMPLE: <code>regmatches(x, gregexpr("\\d+", x))</code> → list of matched digit strings                        |
| <code>strsplit(x, split, fixed = FALSE, perl = FALSE)</code>                 | Splits string vector x into substrings at pattern matches; returns list of string vectors<br><br>EXAMPLE: <code>strsplit("a, b; c", "[,;]\\s*")[[1]]</code> → <code>c("a", "b", "c")</code>                                |

# stringr / Tidyverse Functions (ICU Engine)

| PATTERN / SYNTAX                                                                                  | DESCRIPTION & EXAMPLE                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>str_detect(string, pattern, negate = FALSE)</code>                                          | <p>Detects pattern matches in string vector; returns logical vector (TRUE/FALSE)</p> <p>EXAMPLE: <code>str_detect(c("apple", "banana"), "^a")</code> → <code>c(TRUE, FALSE)</code></p>                                                   |
| <code>str_which(string, pattern) / str_subset(string, pattern)</code>                             | <p>Returns matching indices (<code>str_which</code>) or filtered subset of matching string values (<code>str_subset</code>)</p> <p>EXAMPLE: <code>str_subset(c("dog", "cat", "deer"), "^d")</code> → <code>c("dog", "deer")</code></p>   |
| <code>str_count(string, pattern)</code>                                                           | <p>Counts the number of non-overlapping matches in each string element</p> <p>EXAMPLE: <code>str_count("banana", "a")</code> → 3</p>                                                                                                     |
| <code>str_locate(string, pattern) / str_locate_all(string, pattern)</code>                        | <p>Returns start and end character indices of first match (matrix) or all matches (list of matrices)</p> <p>EXAMPLE: <code>str_locate("hello 123", "\\d+")</code> → start: 7, end: 9</p>                                                 |
| <code>str_extract(string, pattern) / str_extract_all(string, pattern)</code>                      | <p>Extracts first matched substring (character vector) or all matches (list of character vectors)</p> <p>EXAMPLE: <code>str_extract_all("2026-08-19 and 2027-01-01", "\\d{4}-\\d{2}-\\d{2}")</code></p>                                  |
| <code>str_match(string, pattern) / str_match_all(string, pattern)</code>                          | <p>Extracts full matches and each capturing group into columns of a character matrix</p> <p>EXAMPLE: <code>str_match("user: 1042", "(\\w+): (\\d+)")</code> → matrix with full match and group columns</p>                               |
| <code>str_replace(string, pattern, replacement)</code>                                            | <p>Replaces first match with replacement string (supports backreferences \\1, \\2 — NOT \$1)</p> <p>EXAMPLE: <code>str_replace("price: \$10", "\\\$(\\d+)", "€\\1")</code> → "price: €10"</p>                                            |
| <code>str_replace_all(string, pattern, replacement)</code>                                        | <p>Replaces all matches; replacement can be a string or a named vector <code>c('pat1' = 'rep1')</code></p> <p>EXAMPLE: <code>str_replace_all("a 1 b 2", c("a" = "A", "b" = "B"))</code> → "A 1 B 2"</p>                                  |
| <code>str_remove(string, pattern) / str_remove_all(string, pattern)</code>                        | <p>Removes first or all occurrences of pattern (equivalent to <code>str_replace</code> with empty replacement)</p> <p>EXAMPLE: <code>str_remove_all("hello 123 world 456", "\\d+\\s*")</code> → "hello world"</p>                        |
| <code>str_split(string, pattern, n = Inf, simplify = FALSE) / str_split_1(string, pattern)</code> | <p>Splits strings into substrings at pattern matches; returns list, matrix (<code>simplify=TRUE</code>), or vector (<code>str_split_1</code>)</p> <p>EXAMPLE: <code>str_split_1("a,b,c", ",")</code> → <code>c("a", "b", "c")</code></p> |

# Flags, Modifiers & Pattern Options

| PATTERN / SYNTAX                                                     | DESCRIPTION & EXAMPLE                                                                                                                                                                                                        |
|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>perl = TRUE</code> (Base R)                                    | Enables PCRE (Perl-Compatible Regular Expressions) engine; enables lookarounds, non-greedy, named groups<br><br>EXAMPLE: <code>grepl("(?&lt;=id:)\d+", "id:42", perl = TRUE) → TRUE</code>                                   |
| <code>fixed = TRUE</code> (Base R & <code>stringr::fixed()</code> )  | Matches pattern as exact literal byte/character sequence; skips regex parsing for maximum speed<br><br>EXAMPLE: <code>sub(".", "_", "a.b.c", fixed = TRUE) → "a_b.c"</code>                                                  |
| <code>ignore.case = TRUE</code> (Base R)                             | Performs case-insensitive matching across base R functions ( <code>grep</code> , <code>grepl</code> , <code>sub</code> , <code>gsub</code> )<br><br>EXAMPLE: <code>grepl("apple", "APPLE", ignore.case = TRUE) → TRUE</code> |
| <code>useBytes = TRUE</code> (Base R)                                | Performs byte-by-byte matching instead of character-by-character; faster on ASCII/raw bytes<br><br>EXAMPLE: <code>grep("foo", text_vec, useBytes = TRUE)</code>                                                              |
| <code>regex(..., ignore_case = TRUE)</code> ( <code>stringr</code> ) | Case-insensitive modifier for <code>stringr</code> patterns<br><br>EXAMPLE: <code>str_detect("HELLO", regex("hello", ignore_case = TRUE)) → TRUE</code>                                                                      |
| <code>regex(..., multiline = TRUE)</code> ( <code>stringr</code> )   | Multiline mode: <code>^</code> and <code>\$</code> match start/end of each individual line within the string<br><br>EXAMPLE: <code>str_extract_all(text, regex("^\\w+", multiline = TRUE))</code>                            |
| <code>regex(..., dotall = TRUE)</code> ( <code>stringr</code> )      | Dot-all mode: dot ( <code>.</code> ) matches any character including newline ( <code>\n</code> )<br><br>EXAMPLE: <code>str_extract(html, regex("&lt;div&gt;.*?&lt;/div&gt;", dotall = TRUE))</code>                          |
| <code>regex(..., comments = TRUE)</code> ( <code>stringr</code> )    | Verbose mode: ignores whitespace and enables inline comments starting with <code>#</code> inside pattern<br><br>EXAMPLE: <code>regex("\\d{4} # year", comments = TRUE)</code>                                                |
| <code>coll(pattern, locale = 'en')</code> ( <code>stringr</code> )   | Locale-aware collation matching according to standard international sorting rules<br><br>EXAMPLE: <code>str_detect("resume", coll("résumé", ignore_case = TRUE))</code>                                                      |

# R-Specific Syntax, POSIX Classes & Escapes

| PATTERN / SYNTAX                                                        | DESCRIPTION & EXAMPLE                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "\\d", "\\w", "\\s", "\\b"                                              | Double backslashes in standard R strings: R string parser requires escaping the backslash itself<br><br>EXAMPLE: <code>grep1("\\d+", "123")</code> vs <code>r"(\d+)"</code>                                                                    |
| <code>r"(...)"</code> , <code>r"-(...)-"</code> , <code>r"{\d+}"</code> | Raw string literals (R 4.0.0+): No backslash escaping required; delimiters <code>()</code> , <code>[]</code> , <code>{}</code> , or <code>-()</code> -<br><br>EXAMPLE: <code>r"(\b[a-zA-Z0-9._%+~]@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\b)"</code>     |
| <code>[:digit:]</code> , <code>[:alpha:]</code>                         | POSIX character classes: Digits [0-9] and Alphanumeric characters [a-zA-Z]<br><br>EXAMPLE: <code>gsub("[:digit:]", "#", "a1b2")</code> → <code>"a#b#"</code>                                                                                   |
| <code>[:alnum:]</code> , <code>[:punct:]</code>                         | POSIX character classes: Alphanumeric characters and Punctuation marks<br><br>EXAMPLE: <code>gsub("[:punct:]", "", "Hello, World!")</code> → <code>"Hello World"</code>                                                                        |
| <code>[:space:]</code> , <code>[:blank:]</code>                         | POSIX character classes: Whitespace [ \t\r\n\v\f] and Blank characters (space/tab only)<br><br>EXAMPLE: <code>strsplit(text, "[:space:]+")[[1]]</code>                                                                                         |
| <code>[:lower:]</code> , <code>[:upper:]</code>                         | POSIX character classes: Lowercase letters [a-z] and Uppercase letters [A-Z]<br><br>EXAMPLE: <code>grep("^[[:upper:]]", c("John", "jane"), value = TRUE)</code> → <code>"John"</code>                                                          |
| <code>[:xdigit:]</code> , <code>[:cntrl:]</code>                        | POSIX character classes: Hexadecimal digits [0-9a-fA-F] and Control characters<br><br>EXAMPLE: <code>grep1("^[[:xdigit:]]+\$", "1A4F")</code> → <code>TRUE</code>                                                                              |
| <code>(?&lt;name&gt;...)</code>                                         | Named capture group in PCRE (perl = TRUE) and stringr; accessible by column name or group name<br><br>EXAMPLE: <code>str_match("2026-08-19", "(?&lt;y&gt;\\d{4})-(?&lt;m&gt;\\d{2})-(?&lt;d&gt;\\d{2})")</code>                                |
| <code>\\1</code> , <code>\\2</code>                                     | Numbered backreferences in replacement strings (sub, gsub, and stringr all use <code>\\1</code> , <code>\\2</code> — R engines do not support <code>\$1</code> )<br><br>EXAMPLE: <code>gsub("(\\w+)\\s+(\\w+)", "\\2, \\1", "John Doe")</code> |
| <code>(?&lt;=prefix)</code> , <code>(?&lt;!prefix)</code>               | Positive and negative lookbehinds (requires perl = TRUE in Base R or stringr)<br><br>EXAMPLE: <code>str_extract("price: \$42", "(?&lt;=\\\$(\\d+))")</code> → <code>"42"</code>                                                                |